

Top epigramas

Ramírez Ríos, Víctor Hugo
a221205528@unison.mx

Septiembre 2023

1. (read-char* input) Lee una letra de la entrada dada.
2. (peek-char* input) Lee una letra sin descartarla del flujo de la entrada.
3. (char-digit? ch) Verifica si el caracter ch es un dígito. Aquí hay un bug ya que se debe de verificar antes si el caracter es un objeto end of file antes de verificar si está dentro del rango de caracteres dado.
4. (char-delimiter? ch) Verifica si el caracter es un objeto end of line o un espacio o un parentesis abierto o cerrado.
5. (define lex-path «unknown¿”) Define una dirección del lexer como ¿unknown¿que representa el origen.
6. (define lex-line 0) Define las líneas del lexer como 0.
7. (define lex-col 0) Define las columnas del lexer como 0.
8. (struct token (type value line col)) Declara una estructura token la cual va almacenar el tipo, valor, número de líneas y columnas.
9. (define (token-open-paren)) Usando la estructura previamente definida declara el token para los parentesis abiertos.
10. (define (token-close-paren)) Lo mismo pero para parentesis cerrado.
11. (define (token-binop type)) Lo mismo pero ahora se ocupa mandar un tipo ya que que definimos diferentes tipos de operaciones binarias
12. (define (token-number value col)) Define el token para los números con un valor y una columna que se cambian por los default de la estructura.
13. (define (token-number/+ t)) Define el token number positivo. Aquí hay un bug, ya que el nombre de la variable token ignora el ámbito de token que se ha usado para actualizar los valores de los tokens.
14. (define (token-number/- t)) Define el token number negativo. Similar al positivo pero el valor del token es negativo. Mismo bug que la función anterior.
15. (define (token-identifier symbol col)) Define el token symbol
16. (define (token-define col)) Define el token define.
17. (define (lex-open-paren chars)) Define la función que actualiza los valores del token que representa el parentesis abierto. Lee todos los caracteres del stream.
18. (define (lex-close-paren chars)) Lo mismo pero con los parentesis cerrados.
19. (define (lex-whitespace chars)) Actualiza los valores si hay un espacio en blanco o un salto de línea. Si es un espacio en blanco aumenta la columna. Si es un salto de línea aumenta la línea y empieza la columna desde 0.

20. (define (lex-sum-or-number chars)) Actualiza los valores si el char es numero o un signo positivo. Si es un signo positivo lo lee para representar la operación, aumenta la columna y lee los tokens que siguen. Si es un numero llama a otra función que lee todos los números que están consecutivamente hasta que se terminen.
21. (define (lex-negative-number chars)) Similar que el de los números positivos pero este lexer no tiene para representar la operación de resta.
22. (define (lex-mult chars)) Actualiza los valores del token del símbolo de multiplicación, asegurándose que el símbolo este en una posición válida.
23. (define (lex-identifier-or-keyword chars)) Verifica si es un identificador o un keyword. En este caso el identificador sería cualquier variable que este en el regex de `[xyz][xyz1-9]*`, y la única keyword aceptable es el `define`. Aquí hay un bug, ya que se lee los caracteres para que la función `read` pueda leerlos devuelta, pero lo que queremos es reducirlo a solo los caracteres, para poder identificar cadenas como el `define`. Entonces, cambiamos el `read` por el `display`.
24. (define (lex-plain-number chars)) Verifica si los caracteres es un numero y actualiza los valores del token.
25. (define (lex-end-of-file chars)) Cierra el input port y vacía el flujo de datos.
26. (define (signal-lex-error message line col)) Mensaje de error que usa el conteo de líneas y columnas para identificar donde esta el error.
27. (define (lex chars)) Checa todos los posibles tipos de token que puede ser el caracter y llama a su respectiva función.
28. (define (lex-from-file path)) LLama al lexer desde un archivo.
29. (define (lex-from-string str)) LLama al lexer desde un string.